

Integrating Imitation Learning, Real-Time RL, and World Models for Robotic Manipulation

Introduction: We propose a comprehensive project to fuse three cutting-edge approaches for robotic manipulation: **(1)** a Vision-Language-Action (VLA) agent trained via imitation learning for broad task generalization and language understanding, **(2)** real-time reinforcement learning (RL) to boost success rates and adaptability, and **(3)** learned world models to simulate experiences in parallel and accelerate training. Individually, each approach addresses different challenges – VLAs like $\pi_{0.5}$ (pi0.5) excel at open-world generalization but rely heavily on human demonstrations ¹ ², pure RL methods can surpass imitation policies in final accuracy but typically require extensive real-world trial-and-error ³, and world models promise to reduce real-world training by “imagining” outcomes without costly physical trials ⁴. By **combining** these strengths, our goal is to dramatically reduce dependence on human-provided demos while achieving robust, real-world performance on complex manipulation tasks. The following is a step-by-step technical plan for this project.

Step 1: Define Task Scope, Environment, and Base Models

1.1 Specify Target Tasks and Setup: We begin by defining the manipulation tasks and the robotic platform. For example, tasks could include multi-step household chores (cleaning a table, arranging objects, etc.) in varied, unstructured environments (different rooms with random object configurations). Clearly define success criteria for each task (e.g. object moved to target location, spill wiped up, etc.) and how to measure it. We assume access to a physical robot (or robots) with the necessary sensors (RGB-D cameras) and effectors, as well as a reset mechanism or human operator to assist if the robot gets stuck or the task needs resetting between trials. We also prepare a simulation or controlled lab environment for initial experiments to ensure safety before full real-world trials.

1.2 Select a Vision-Language-Action Base Model: Choose an imitation-learning based VLA architecture as the starting policy. A strong candidate is $\pi_{0.5}$ (**pi0.5**) – a state-of-the-art Vision-Language-Action model known for its open-world generalization ¹. Pi0.5 was co-trained on heterogeneous data (including image-caption pairs, human instructions, and multi-robot demonstrations) to imbue it with broad visual-semantic understanding and basic skills ². Using such a foundation is crucial because it provides the agent with **general knowledge** (e.g. how to grasp common objects, the semantics of instructions) out-of-the-box. If pi0.5 (or a similar model) is available open-source, we will incorporate it; otherwise, we will implement a comparable VLA model (for instance, by fine-tuning a large vision-language transformer on robot demonstration data and instruction following). The chosen model should accept visual observations and a language instruction as input and output high-level action commands or low-level controls. We will integrate it with the robot's control interface (e.g. via ROS or direct joint velocity commands) and verify it can run in real-time on available hardware (using GPU acceleration for the model inference if needed).

1.3 Baseline Data and Performance: Assemble any initial datasets needed to bootstrap the model. This includes: a moderate number of **human teleoperated demonstrations** for the target tasks (if available), and possibly additional broad datasets (the same types used to train pi0.5: web images with captions,

question-answer pairs, scripted robot trajectories, etc.) to ensure the model has rich prior knowledge. We will define a baseline by evaluating the pre-trained VLA agent on the tasks **before RL fine-tuning** – this gives an initial success rate and behavior profile. From prior work, we expect the imitation-trained VLA to exhibit **strong generalization** (it can attempt tasks in novel scenes it was never explicitly trained on) but not necessarily achieve high success on first try ⁵. This baseline will guide our expectations and help measure improvement after incorporating RL and world models.

Step 2: Pre-Train the VLA Agent via Imitation Learning

2.1 Diverse Multimodal Training: Using the data from Step 1.3, we train or fine-tune the VLA policy with *imitation learning*. The training objective is to mimic demonstrator actions given observations and instructions (behavioral cloning). We leverage **heterogeneous data sources** as in the $\pi_{0.5}$ recipe, mixing multiple types of supervision to encourage both low-level skill learning and high-level reasoning ². For example, the training mix can include: (a) image-text pairs and question-answer data to teach visual-semantic comprehension, (b) teleoperation trajectories of manipulation tasks to teach motor skills, (c) high-level descriptions paired with observations (e.g. an image of a messy room labeled “tidy up by putting items in the basket”) to teach semantic task understanding, and (d) step-by-step language instructions aligned with demonstration steps to teach following natural language commands. This multimodal co-training approach was key to $\pi_{0.5}$ ’s ability to infer subtask structure and respond to novel instructions ² ⁶. We will reproduce a similar training curriculum, possibly in two stages: first training a visual-language model on general data (for perception and language grounding), then fine-tuning it on robot demonstration sequences (for action generation).

2.2 Leverage Pre-trained Models: To expedite training, we initialize the agent’s visual encoder and language encoder with weights from large pre-trained models (e.g. a vision transformer trained on ImageNet and a language model or multi-modal transformer trained on internet data). This transfer learning allows the agent to **inherit “common-sense” knowledge** about the world ⁷. Pi0.5 itself was built on top of a pre-trained vision-language model, which gave it semantic reasoning abilities from internet-scale training ⁷. We will mirror this strategy so that the agent knows, for instance, what a “spatula” or “plate” is, even before any task-specific training – an ability crucial for open-world performance. After initializing, we train the combined VLA model on our task data with imitation loss (and potentially auxiliary losses for any predictive tasks, similar to how $\pi_{0.5}$ uses a flow model for low-level action consistency ⁷). Training is done until the policy performance converges on the demonstration distribution (e.g. low imitation loss on validation demonstrations). We will monitor that the model not only copies actions but also *correctly interprets the language instructions and visual context*, which is essential for later zero-shot generalization.

2.3 Validate Generalization Capability: Once trained, we evaluate the VLA agent in a few *unseen environments* (e.g. a new room arrangement or objects not present in training) to ensure it exhibits the intended generalization. We expect behavior akin to what the $\pi_{0.5}$ reports described: the agent should attempt reasonable actions in novel scenarios, perhaps with some creativity or “chain-of-thought” reasoning ($\pi_{0.5}$ could break high-level commands into subgoals internally, like converting “clean the bedroom” into steps such as “pick up the pillow” ⁸). For example, if instructed to “set the table” in a new kitchen, the VLA might identify plates on a shelf and attempt to place them on the table – even if the exact shelf or plates were never seen during training. This generalization check is important: it confirms that our imitation learning phase has yielded a policy with broad task understanding, forming a strong foundation for the reinforcement learning phase.

Step 3: Set Up the Real-Time Reinforcement Learning Pipeline

3.1 Reinforcement Learning Framework Selection: With a competent initial policy in hand, we design an RL algorithm that will fine-tune this policy on real robot interactions to boost its success rate and adaptability. We opt for an *off-policy* actor-critic method augmented for sample-efficiency, since real robot data is expensive. In particular, we choose a variant of the **Dreamer** algorithm (which uses a world model – to be introduced in Step 5) for its efficiency in learning from pixels ⁹. Dreamer has been shown to outperform pure model-free RL in simulation by learning from imagined trajectories ⁹, and its adaptation to real robots (DayDreamer) proved effective without simulations ¹⁰. Alternatively, we consider a simpler off-policy RL like Soft Actor-Critic (SAC) with *experience replay*, combined with demonstrations (for a jumpstart) and human interventions. Ultimately, the pipeline will involve collecting data with the real robot, updating the policy, and iteratively improving performance – all **online** (in real time as the robot trains). We ensure we have the computing setup to run training concurrently with the robot operation (e.g. a computer with GPU that can receive camera input and update the model on-the-fly).

Figure: An example human-in-the-loop RL training workflow. Initially, a few demonstrations and a learned reward model guide the robot. During online training, the robot practices the task via trial-and-error, and a human provides occasional corrective interventions to prevent failures and demonstrate subgoals. Over time, the policy requires fewer interventions as it becomes proficient ³.

3.2 Reward Function Design: Define a **reward signal** for each task to drive RL. Since tasks are specified by language (e.g. “pour water into the cup”), designing an automatic reward can be challenging – we may need to train a reward classifier or use task-specific sensors. We follow the **HIL-SERL** approach: first, use the teleoperated data from Step 1.3 to train a binary reward classifier that recognizes successful outcomes vs. failures from observations ³. For instance, we collect positive samples (successful end states) and negative samples (incorrect outcomes) for the task and train a vision model to output a reward signal. This learned reward function allows sparse rewards even when explicit numeric rewards are not readily available. In tasks where a straightforward metric exists (like distance to a target position), we can combine that with the learned signal. The key is to have a **sparse but informative reward** that eventually aligns with task success.

3.3 Human Interventions for Safety and Exploration: Especially in early training, to prevent the robot from catastrophic failures (knocking objects off the table, getting stuck, etc.) we include a **human-in-the-loop** component. During policy rollouts, if the robot is about to do something harmful or obviously wrong, a human operator will intervene – either manually taking over control to correct the robot’s pose or giving a corrective demonstration from that state ¹¹. We plan an intervention protocol: initially, frequent interventions to keep the robot on track and show it how to recover, and gradually reducing help as the policy improves (mirroring the HIL-SERL method ¹²). These interventions will also be added to the replay buffer as additional “on-policy” demonstration data, which helps the agent learn the correct behavior from states it might otherwise struggle with. By handling unsafe or tricky scenarios in this guided manner, we maintain safety and also enrich the training data with solutions for those scenarios.

3.4 Replay Buffer and Training Loop: Set up a replay buffer to store trajectories comprising: state observations (camera images, proprioception), the instruction, actions taken, and rewards obtained. We initialize this buffer with: (a) the small set of demonstration trajectories (from Step 1.3) marked with high reward (so the agent has some successful examples to start), and (b) possibly some *exploratory data* where the agent acted randomly or using the initial policy, to provide a range of experiences. During training, the

robot will continuously collect new data: for each episode, it receives an instruction, attempts the task using its current policy (with added exploration noise), and the outcomes and rewards get added to the buffer. The RL algorithm then samples mini-batches from this growing buffer to update the policy network (and value network if using actor-critic). We will use a relatively high update frequency (many gradient steps per real-world episode) to fully leverage the collected data, as is common in off-policy RL. This real-time learning setup ensures that **every bit of real experience is squeezed for maximum learning**. With the initial demo data present, the agent won't start from scratch – it will initially emulate the demo behavior (thanks to either pre-training or the demo buffer) and then gradually improve beyond it.

Step 4: Integrate Imitation Policy with Reinforcement Learning

4.1 Policy Initialization: Rather than training the RL policy from a blank slate, we initialize the actor (policy) network with the weights of the pre-trained VLA model from Step 2. This means at the start of RL, the policy already knows how to perform the task *in principle* (as learned via imitation) and how to interpret the language instructions. By doing this, we leverage **the generalization and knowledge of the IL model** and save a huge amount of interaction data – the policy doesn't need to rediscover basic behaviors. Concretely, the vision and language encoders of the policy network are frozen or partially frozen to retain their pre-trained features, and the action output layers start from the pre-trained weights but remain trainable to adapt to the reward. This provides a strong prior. For example, if the instruction is “pick up the blue cube,” the policy already can identify the blue cube and roughly how to grasp it due to imitation learning; RL will then refine *how exactly* to grasp it to maximize reward (success rate).

4.2 Hybrid Loss (IL + RL): During the initial phase of RL training, we may apply a *behavior cloning loss* on demonstration transitions in addition to the RL loss (actor-critic or Q-learning loss). This technique, often called **DQfD (Deep Q-learning from Demonstrations)** or **behavior cloning assisted RL**, helps stabilize training and prevents the policy from drifting too far from reasonable behavior early on. Practically, our loss function is $L = L_{RL} + \lambda * L_{IL}$, where L_{RL} is the standard reinforcement learning loss (negative expected reward or temporal-difference error, etc.) and L_{IL} is a supervised imitation loss on the demo actions, with λ decaying over time. This ensures that initially the policy stays close to imitation (for safety and efficiency), and as it learns from reward, it gradually relies more on its own experience. HIL-SERL effectively did this by seeding the replay buffer with demos ³; we optionally reinforce it with an explicit loss on those samples as well.

4.3 Language-Conditioned RL: Because our tasks are specified via language and the policy takes language input, we ensure the RL algorithm properly handles this context. One approach is to concatenate the instruction embedding to the state vector that goes into the policy network and value network. The policy then learns a **contextual policy** $\pi(a | s, \text{instruction})$. We will train on potentially multiple tasks or variations concurrently (multi-task RL), distinguished by the instruction. The reward function will be aligned with the instruction as well – e.g., success is only rewarded for completing the commanded task. This setup enables *instruction-conditioned RL fine-tuning*: the agent should learn to complete whichever task is asked, using the language understanding from the VLA pre-training to know what the goal is. In evaluation, this allows zero-shot generalization to new instructions *within the training scope* of capabilities. (If a new instruction describes a novel combination of known skills, the agent's pre-trained language understanding should help interpret it correctly – and the RL fine-tuning will have taught the agent to trust that language input as defining the goal). This tight integration of language in the RL loop is relatively new, and we will carefully monitor that the agent doesn't overfit to specific phrasing; we might augment training with paraphrased instructions for robustness.

Step 5: Develop a Learned World Model for Parallel Simulation

5.1 World Model Architecture Selection: A core innovation in this project is incorporating a **world model**, which is a predictive model of the robot's environment and dynamics. We consider two state-of-the-art approaches for visual world models: - **Latent Dynamics Model (DreamerV3-style):** Learn a compact latent state representation (using a variational encoder for images and perhaps text), and train a recurrent dynamics model in that latent space to predict future states and rewards. This model can imagine trajectories much faster than real-time since it operates in latent space ⁴. DreamerV3's architecture (CNN encoder, GRU or transformer dynamics, MLP decoder for reward) is a strong candidate. - **Video Generation Model (Diffusion or JEPA):** Alternatively, use a generative model that predicts the *next image frames* given the current frame and action. For example, **UVA (Unified Video Action model)** by Stanford integrates video generation with policy learning, using diffusion models to predict future frames ¹³. Another example is Meta's **V-JEPA 2**, a joint-embedding predictive architecture trained on internet-scale videos (over 1 million hours) to understand physical dynamics ¹⁴. V-JEPA 2 is optimized for speed, reportedly 30× faster than previous models ¹⁵, making it suitable for real-time rollout. We will evaluate these options; a latent model is simpler and proven in Dreamer, whereas a video predictive model might leverage large-scale pretraining. A hybrid is possible: e.g., use V-JEPA's pre-trained representation as the latent state for a Dreamer-like dynamics model. The chosen world model will accept the current state (image and possibly proprioception) and an action, and output a prediction of the next state (or its latent encoding) and reward.

5.2 Training the World Model: We will train the world model **in parallel** with the policy. Initially, we can pre-train it on any available interaction data (the demonstration set and some random exploration data) to give it a head start. If using a pre-trained model like V-JEPA 2, we fine-tune it on our robot's data to specialize it (for example, align its predictions with the camera viewpoint and robot dynamics of our setup). The world model learning objective is to minimize prediction error: it tries to predict the sequence of states and rewards observed in the real world. We will use standard loss functions like mean squared error for latent predictions or image pixel loss (possibly with perceptual metrics if using raw image prediction). The **training is online:** as new episodes come in, we continually update the world model on these new transitions (using a short window of recent data to adapt to changes). We keep an experience dataset for the world model (this can be the same as the replay buffer, though often world model training uses the latest data to avoid stale dynamics). Over time, the world model becomes an increasingly accurate simulator of the robot's environment dynamics. Notably, our world model will incorporate **visual context and potentially language:** the state representation can include the image of the scene and we may condition it on the task instruction if the task changes the dynamics (the instruction might not directly affect physics, but it can influence which aspects of the scene are relevant). Indeed, the UVA authors noted that integrating diverse language inputs into world models could help the model handle novel tasks better ¹⁶ – so we plan to provide the instruction as an additional input to the world model network, enabling it to focus predictions on goal-relevant aspects (for example, predicting what happens when trying to “open the drawer” versus “push the chair” might emphasize different parts of the scene).

5.3 Faster-Than-Real Simulation: The appeal of world models is that they can be used to simulate many trials quickly. Once our world model achieves reasonable fidelity (even if not perfect, it should capture the gist of object motions and task outcomes), we will use it as a *parallel simulator*. This means generating synthetic rollouts of the robot performing tasks in its “**dream**” environment. Since it's just a computation, we can run it faster than real-time and even in parallel threads. For example, while the physical robot performs one attempt, the world model could simulate dozens of alternative attempts with different actions. Modern world models like V-JEPA 2 are optimized for speed, enabling real-time or faster predictions

¹⁵, so with a GPU we might run, say, 10 imagined rollouts in the time the robot does 1 real rollout. We will configure a pool of parallel processes (or a vectorized model that can simulate multiple trajectories simultaneously in batch) to fully exploit this. The result is an abundance of *additional training data* for the RL algorithm without wearing out the real robot. These imagined trajectories will be treated similarly to real ones in training the policy, except we must be mindful of model bias (imperfect world model predictions). We will mitigate that by continuously updating the world model with new real data so it remains accurate, and possibly by mixing a higher proportion of real data vs. imagined data in the replay buffer if we detect divergence.

Step 6: Integrate World Model with Policy Training (Parallel Training)

Figure: Integrating a world model (Dreamer approach) for parallel simulation. The agent collects real experiences into a buffer, trains a predictive model of the world from those experiences, and then uses the model to imagine future trajectories. These imagined trajectories are used to train the policy (and value) networks, allowing the agent to learn from many more trials than it physically experiences ⁴.

6.1 Asynchronous Training Loop: We will run an **asynchronous dual-loop** training process inspired by DayDreamer and Dreamer. One thread (the “Real Robot Loop”) continuously runs on the robot: it resets the environment (or positions for the task), feeds the current policy and instruction to generate actions, executes them, and logs the results (observations and rewards) to the replay buffer. Another thread (the “Training Loop”) continuously updates the agent using both real and imagined data: - **World Model Update:** Frequently (e.g. every few episodes or continuously in background), train or fine-tune the world model on the latest data from the replay buffer so that its predictions stay in sync with the real environment’s dynamics. - **Imaginary Rollouts:** Using the current world model, generate simulated rollouts. For each rollout, we sample a recent real state as a start (or use the initial state of a real episode), then let the policy interact with the world model for H steps into the future. Because the world model can generate next-state predictions quickly, we can simulate H-step trajectories rapidly. During these imaginary rollouts, we query the world model’s predicted reward at each step (or at least the final step if only final reward matters). We store these imagined transitions (state, action, “next state”, reward) into a **separate imagination buffer**. - **Policy Update:** Sample batches from a mixture of (a) real transitions from the real replay buffer and (b) imagined transitions from the imagination buffer. Use these to update the actor and critic networks via the chosen RL algorithm (for Dreamer, this involves propagating gradients through the imagined sequence to update an actor; for a simpler approach, we could treat imagined data like additional off-policy data for a value-based learner). The key is that the policy learns not just from what *has been tried* in reality, but also from what *could have been tried*, as per the world model’s predictions.

This asynchronous design ensures that **while the robot is busy doing one thing, the GPU is busy training on a hundred other hypothetical experiences**, thereby massively accelerating learning. DayDreamer demonstrated the power of this: using such imagination, a robot could learn tasks like pick-and-place directly from camera images and *approach human performance* within a reasonable time ¹⁷, and a quadruped adapted to perturbations within minutes ¹⁸, something infeasible with sparse real trials alone. We aim to replicate this kind of efficiency.

6.2 Dealing with Model Imperfections: We acknowledge that early in training, the world model will be approximate – if misused, imagined data could corrupt training (e.g. the agent might exploit world model

errors that don't work in reality). To handle this, we will: - Limit the **imagination horizon** H initially (e.g. 5-10 steps) to reduce compounding prediction errors. As the world model improves, we can gradually extend H . - Use **conservative mixing ratios**: in early stages, rely more on real data for policy updates, and slowly increase the fraction of imagined data as we trust the model more. For instance, start with 80% real, 20% imagined data in updates; later shift to 50-50 or more imagined. - Employ techniques like **model uncertainty** estimation: if available, have the world model indicate uncertainty in its predictions (via an ensemble or variance in an output distribution). We can then down-weight or reject imagined trajectories where the model is very uncertain (to avoid learning from highly speculative scenarios). - Keep the human in the loop for safety: If the policy, under the influence of imagined data, starts suggesting actions that are dangerous or clearly wrong, the human interventions from Step 3.3 will kick in to prevent damage, and those states will be corrected with proper examples – thus the real data will counteract any world model hallucinations over time.

By carefully managing these aspects, we ensure that the **world model serves as a helpful coach** that broadens the agent's experience, rather than leading it astray. The net effect is that the agent can achieve proficiency with far fewer real-world trials than would otherwise be required, aligning with Meta's vision that world models enable learning without astronomical amounts of physical data ¹⁹.

Step 7: Real-World Training and Iterative Improvement

7.1 Executing Training Episodes: With all components in place (initialized policy, reward mechanism, world model, etc.), we launch the training on the real robot. The process will be iterative over many episodes (each episode being one attempt of the task or a fixed time horizon). We will monitor key metrics in real-time: success rate of each trial, cumulative reward, any safety triggers (interventions), and so on. Early on, the success rate may be modest (the imitation policy might succeed sometimes but fail in others, especially in new environments). As training progresses, we expect to see rapid improvement due to the RL fine-tuning and world-model-assisted data augmentation. For example, HIL-SERL's results showed the policy's success climbing significantly with just a few hours of training ²⁰; similarly, DayDreamer's robots learned skills in under an hour to human-level performance ¹⁷. We anticipate on the order of tens to low-hundreds of real episodes (potentially collected over a day or two of real time, depending on episode length and parallelization) may be sufficient to reach high success rates, thanks to the parallel "dreamed" training.

7.2 Ongoing Human Role: As training continues, the human supervisor will steadily reduce their involvement. Initially, we might intervene in, say, 30% of episodes to prevent bad outcomes; but by later stages, interventions might drop near 0% as the policy becomes reliably safe and competent ²¹. We will formally schedule evaluation checkpoints (e.g. after every N episodes, run 10 trials with no human help to gauge true autonomous performance). If the policy reaches a plateau or shows instability, we can decide on-the-fly adjustments: for instance, if we notice the agent struggling with a specific scenario, we can provide a few extra demonstrations for that scenario and inject them into the buffer (a targeted boost of imitation learning). This adaptability is part of the human-in-loop strategy to ensure smooth learning.

7.3 Continual Refinement: The beauty of combining IL, RL, and world models is that training doesn't have to stop at a fixed endpoint – the system can **continuously improve**. We plan the training in phases: an initial phase until the agent achieves, say, ~90% success on training tasks consistently, then an extended phase where we introduce new variations (moving to entirely new environments or asking novel instructions) to evaluate how well the agent adapts. Because of the generalization ability inherited from the VLA and the adaptive RL, the agent should handle many novel scenarios gracefully. If a novel scenario is too

challenging (success drops), the same pipeline (with possibly a few guiding interventions) can be applied to learn it. In essence, our robot will keep learning on the job, getting more robust over time. We will log all new experiences to potentially further train both the policy and the world model offline if needed.

Step 8: Evaluation of Performance and Generalization

8.1 Quantitative Evaluation: After training, we will conduct a thorough evaluation. This includes running the robot for a large number of trials *without any human help* on a variety of test scenarios. We measure **success rate**, **task completion time**, and compare against baselines: - Baseline 1: the *imitation-only* policy (from Step 2) without RL fine-tuning. - Baseline 2: a *pure RL* approach without world model (e.g. if we ablate the imagination component). - Baseline 3: if possible, *human performance* on the task or an approximate upper bound (for instance, if humans succeed 100% or take X seconds on average, etc.).

We expect the fused approach to significantly outperform Baseline 1 (since pure IL might have lower success in new envs ⁵). Compared to Baseline 2, our approach should achieve similar or better success *with far less real data*, demonstrating sample efficiency. For example, if HIL-SERL needed X hours to reach Y% success, we might reach the same Y% in a fraction of that time due to the world model’s contribution. Success rates approaching human-level were reported in prior work for pick-and-place tasks using world-model RL ¹⁷, so we aim for similarly high performance. We will cite concrete numbers in our report, e.g., “The agent achieved ~95% success in cleaning tasks across 5 unseen homes, compared to 60–70% for the initial imitation policy.” High success in varied environments will validate robust real-world performance.

8.2 Qualitative Analysis: We will also analyze *how* the agent is accomplishing the tasks. Thanks to the language component, we can issue completely new instructions and observe if the agent can perform zero-shot new behaviors. For instance, if we never explicitly trained “water the plant” but the knowledge exists in VLA from web data, can the robot figure it out with minimal adaptation? We’ll document cases of creative problem-solving or multi-step reasoning the robot exhibits (e.g., the agent might verbalize a chain-of-thought internally if that’s part of the model – $\pi_{0.5}$ was noted to exhibit a form of chain-of-thought via high-level action predictions ²²). We’ll also verify the agent’s resilience: if something unexpected happens (a human moves an object mid-task), does the policy adapt on the fly? The real-time RL training likely endows it with more **adaptability to perturbations**; recall how Dreamer-trained robots adapted within minutes to pushes on a quadruped ¹⁸. We may perform perturbation tests to confirm this adaptiveness. Any failure cases will be examined to identify if it’s a limitation of the world model (e.g., it didn’t predict a certain outcome) or policy (e.g., it couldn’t reason about a new object). This analysis will feed into further improvements.

8.3 Generalization to New Tasks: A key outcome we expect is reduced need for demonstrations when tackling *new but related tasks*. To test this, we can introduce a new task instruction after the initial training (for example, if we trained on “clean the table” and “organize the shelf”, now we ask “set up dinner table” which might involve elements of both). Without providing any demo for this exact task, we let the agent try to learn it via RL (with the world model) from scratch. Because the agent’s VLA core has broad knowledge (like how to pick up plates, where plates belong, etc.), it should learn the new task with **very few trials**, far fewer than a naive agent. We will measure how quickly the agent reaches proficiency on a new task with no human demonstrations – this showcases the goal of *dramatically reduced reliance on human demos*. If the world model is well-trained by prior tasks, it will also facilitate quick learning on the new task by imagining outcomes. Success in such an experiment would underscore the effectiveness of our approach: it would

validate the claim that robots can acquire new skills in the real world with minimal human guidance by leveraging prior knowledge and “dreamed” experience ¹⁰ .

Step 9: Reducing Demonstration Dependence and Scaling Up

9.1 Minimizing Demonstrations: As we progress, we will continuously analyze where human demonstrations are still needed and strive to reduce that. Initially, we used some demos to kickstart imitation learning and reward learning. But one long-term objective is to see if even that could be minimized by using autonomous exploration plus world model learning. For instance, **exploration:** the agent with a language prior might be able to figure out parts of the task by itself (the way children learn by exploration). With a world model, the agent can even generate its own synthetic training goals (e.g., imagine various goal states and practice achieving them in simulation). We will experiment with cutting down the demo data – say, use only half the demonstrations or none at all – and check if the combination of language priors + RL + world model can still learn the task. DayDreamer’s study notably succeeded *without any demonstrations* for their tasks, relying purely on reward and world model ¹⁰ . In our case, the language and vision pre-training provides some prior that partially substitutes for demonstrations. If we can show that an agent can go from “knowing nothing about this specific task” to succeeding, purely through our reinforcement and imagination loop (perhaps with human interventions just to avoid damage), that will be a huge win for scalability. It means future users could deploy the system on new tasks without collecting large teleoperated datasets upfront – the agent would learn by doing, guided by its world-model-enhanced intuition.

9.2 Scaling to Broader Skill Sets: We will also plan for scaling up the number of tasks and skills. Our architecture is inherently multi-task (since the instruction text defines the task), so in principle we can keep adding new task descriptions and let the robot train on them. We might set up a **curriculum:** start with simpler tasks and gradually add complexity (much like how humans learn basics then combine them). Thanks to the modular design, previously learned skills (in policy and world model) will aid new ones. For example, if the agent learned “pick and place objects” and “open a door” separately, it should be able to tackle an instruction that requires both (like “open the cupboard and put the book inside”). We can quantitatively track learning curves as more tasks are introduced in parallel – this will test the **generalist capability** of our VLA+RL agent. We anticipate that our approach, akin to training a “general-purpose home assistant”, will showcase far greater **task diversity** than any single-method system. VLAs trained on internet data provide the breadth of understanding, and the real-time RL provides the depth of skill for each specific task, all accelerated by the world model. This synergy is what could usher in “a new era for robotics”, where robots no longer need colossal task-specific datasets to be competent ¹⁹ .

9.3 Long-Term Autonomy: Finally, we consider the long-term deployment. The trained policy (with world model) can be deployed in a real home or environment for extended periods. It should handle variations and even learn from them in real-time (we could leave the learning switch on so it keeps updating its policy mildly on new data, achieving lifelong learning). The world model will also continue to refine as it sees more situations, becoming more predictive. In essence, the robot becomes a **self-improving system:** the longer it operates, the better it gets, without needing constant human-provided examples. We will set up logging to capture any anomalies or performance drifts and have fallback safety policies if needed (for instance, if the robot encounters something completely outside its experience, it might default to a safe mode or request help). But ideally, with enough breadth in the VLA pre-training and a strong learning loop, the robot can handle most things on its own.

Step 10: Project Outcomes and Future Extensions

By following these steps, we expect to demonstrate a robot manipulation system that leverages *Imitation Learning for generalization*, *Real-Time Reinforcement Learning for high success and adaptability*, and *World Models for data efficiency*. The combined system should achieve robust performance on real-world tasks with far fewer human demonstrations than previous approaches. In summary, imitation-trained VLAs like $\pi_{0.5}$ provided the initial “common sense” and versatility ¹ ², RL fine-tuning (as seen in HIL-SERL) ensured the policy reaches expert-level precision ³, and world model imagination (as in DayDreamer) vastly accelerated learning without needing a simulator or exhaustive trials ¹⁰ ⁴. This fusion addresses the key challenge of generalization in robotics while minimizing human effort.

Moving forward, there are exciting extensions to explore: - **Improving World Model Fidelity:** We could incorporate richer modalities (depth sensing, audio, force feedback) into the world model to better predict complex manipulation physics. The UVA project notes extending to audio or touch is straightforward by adding prediction heads ²³. - **Larger Models and More Compute:** As suggested by UVA and others, scaling up model size and training data can further improve performance ²⁴. We could train larger VLA or world models, given more GPU resources, to see if the robot’s capabilities improve (e.g., handling even more abstract instructions or complicated sequences). - **Transfer to New Robots:** Thanks to the cross-embodiment training data used in $\pi_{0.5}$, our policy representation might transfer to different robot hardware with minimal retraining ²⁵. We can test deploying the learned policy on a different arm or a mobile manipulator, adjusting the low-level control as needed, to evaluate how well the high-level knowledge carries over. - **Zero-Shot Planning:** With a trained world model, we can also attempt pure *planning* in the learned model for new goals (similar to how V-JEPA 2 enables zero-shot planning in novel situations ²⁶). For example, given a goal state description, the agent could use the world model to search for an action sequence achieving it, without additional training. This could make the system even more flexible in solving one-off tasks. - **Community and Benchmarks:** We will contribute our findings and perhaps our combined algorithm back to the research community. A successful demonstration would be a proof-of-concept that combining IL, RL, and world models is the way forward for embodied AI. It would inspire new benchmarks and challenges (as Stanford’s BEHAVIOR or others are pushing for broad task evaluation ²⁷) to test generalist robots in the real world.

In conclusion, this project’s step-by-step plan lays out how to build an advanced robotic manipulation agent that **learns like a human** – drawing on prior knowledge, learning from real-world trial and error, and imagining experiences to refine its skills. By “dreaming” about its tasks with a world model and continuously improving through reinforcement, the robot will require much less hand-holding from humans, yet achieve the kind of generalizable, robust performance that has long been sought in robotics ¹⁹. The end result will be a significant leap towards lifelong learning robots that can be deployed in unstructured real-world environments and progressively learn to handle whatever tasks come their way.

Sources: The concepts and plan are informed by recent advances in Vision-Language-Action models ¹ ², human-in-the-loop robot RL ³, and world model-based learning ⁴ ²⁸, which together point to the potential of this integrated approach. The success of $\pi_{0.5}$ in generalizing to new environments ²⁹, the efficiency of HIL-SERL in achieving high precision via online RL ³, the ability of world models like UVA to boost policy success by imagining outcomes ³⁰, and the real-world learning demonstrated by DayDreamer without simulations or extensive demos ¹⁰ all inspire the components of our plan. By uniting these strands of research, we aim to push the frontier of what robots can do autonomously in the real world.

1 2 5 29 A VLA with Open-World Generalization

<https://www.physicalintelligence.company/blog/pi05>

3 11 12 20 21 HIL-SERL: Precise and Dexterous Robotic Manipulation via Human-in-the-Loop Reinforcement Learning

<https://hil-serl.github.io/>

4 9 10 17 18 DayDreamer: training robots in the real world with online reinforcement learning

<https://www.actuia.com/en/news/daydreamer-training-robots-in-the-real-world-with-online-reinforcement-learning/>

6 25 $\pi_{0.5}$ (Pi05) Policy

<https://huggingface.co/docs/lerobot/en/pi05>

7 [PDF] π_0 : A Vision-Language-Action Flow Model for General Robot Control

<https://www.physicalintelligence.company/download/pi0.pdf>

8 22 [Paper Review] Pi0, Pi0.5, Pi0-FAST - Tracing the Path of Physical ...

<https://bequiet-log.vercel.app/pi-review>

13 16 23 24 30 Unified Video Action Model

<https://unified-video-action-model.github.io/>

14 15 19 28 Meta's V-JEPA 2 model teaches AI to understand its surroundings | TechCrunch

<https://techcrunch.com/2025/06/11/metasp-v-jepa-2-model-teaches-ai-to-understand-its-surroundings/>

26 V-JEPA2 — Yann LeCun's Vision for AI | by Mayur Jain | MLWorks

<https://medium.com/mlworks/v-jepa2-yann-lecuns-vision-for-ai-f69f705a934c>

27 BEHAVIOR Challenge Charts the Way Forward for Domestic Robotics

<https://hai.stanford.edu/news/behavior-challenge-charts-the-way-forward-for-domestic-robotics>